

AI Use Declaration Form

Course: Introduction to Artificial Intelligence

Lab Title: Lab 1 Part A — Uninformed Search Algorithms

Student Name: __Naima Tahirou

Maiyaki_____

Student ID: _____30252027_____

GitHub Repository Link:

__https://github.com/naimatahiroumaiya/Lab_1_____

Date Submitted: _____6/20/2026_____

1. AI Use Summary

Question	Student Response
Did you use any AI tool for this lab?	yes
Estimated percentage of the work influenced by AI	14 %
Did you attach evidence of AI use where applicable?	yes

2. Details of AI Use

Complete the table below for each AI tool used. Add more rows where necessary.

Name of AI Tool Used	Purpose for Using the Tool	Prompt or Instruction Given to the Tool	Part(s) of the Work Influenced by the Tool	How I Verified, Edited, or Improved the AI Output
ChatGPT	To understand search algorithms, debug code, interpret results	I asked ChatGPT to explain the logic of WeightedGridProblem, heuristic functions, Best-First Search, Greedy Search, A*, and Weighted A*. I also requested help debugging errors, analyzing experiment results	Algorithm implementation, debugging, experiment analysis, custom map design, interpretations	I compared the explanations with the lab instructions, and pseudocode. I tested all code in the notebook, verified results using the self-checks and output tables to reflect my own understanding

3. Attachment of AI Output Evidence

Where required, attach evidence of AI use. Tick all that apply. Add more rows where necessary.

Evidence Type	Attached? (Yes, No, N/A)	File Name
AI-generated draft	no	n/a
Prompt history	no	n/a
Screenshot of AI interaction	yes	n/a
Exported AI conversation	no	n/a
AI-generated code snippet	yes	n/a
Revised version showing student input	yes	n/a

4. Student Declaration

I declare that:

Declaration Statement	Response (Yes/No)
The submitted work is my own work.	yes
Any use of AI tools has been clearly declared in this form.	yes
The prompts, instructions, and AI-generated outputs have been disclosed where applicable.	yes
I reviewed, tested, edited, and improved any AI-generated content before submission.	yes
My AI usage does not exceed 25% of the entire work.	yes
I understand that undeclared or excessive AI use may be treated as academic misconduct.	yes

Student Signature: _____ NaimaMaiyaki _____

Date: _____ 6/20/2026 _____

Appendixes:

we will do the same thing we did for the part A understand the lab , then we explain the concepts step by step



Perfect. Since this is the same AI/Search course as Part A, we'll use the same approach:

Step 1: Understand the Purpose of Part B

Before writing any code, we first identify:

1. What problem is the lab solving?
2. What AI concepts are being taught?
3. What new algorithms or techniques are introduced?
4. How do they relate to Part A?

However, I cannot yet see the actual contents of **Lab_1_Part_B.ipynb**. The file search did not retrieve the notebook content itself—it returned unrelated files from your library.

Next Step

I need to open and read the notebook first.

Please either:

- Upload **Lab_1_Part_B.ipynb** again, or
- Tell me specifically

I want to understand the logic do not provide the answer to the code:
3. Weighted Terrain: WeightedGridProblem
In Part A every move cost exactly 1, so the shortest path (fewest moves) was also the cheapest path. Real drone flight is not like that. We now attach a terrain cost to every free cell:

Terrain	Example cost	Meaning
Open ground	1	Calm, easy flying
Forest canopy	3 – 5	Turbulence, slower progress
Water body / wind corridor	5 – 9	Strong winds, high battery drain
Obstacle	1 in grid	Cannot be entered at all

Afficher plus ▾

Excellent. This is exactly the right question. Don't think about the code yet. Let's understand **why this method exists**.

First: What changed from Part A?

In Part A, every move had the same cost.

Imagine this grid:

S . .
. . .
. . G



explain the logic, do not provide answers: 4. Heuristic Functions
A heuristic function $h(n)$ estimates the cost from a node's state to the goal using domain knowledge. For our drone, the domain knowledge is geometry: GPS gives us the coordinates of the current cell and the goal cell.

In this lab a heuristic is a plain function with the signature:

```
def heuristic(state: Tuple[int, int], goal: Tuple[int, int]) -> float
```

You will implement two classic grid heuristics:

Afficher plus ▾

This section is probably the **most important concept in the entire lab**, because it explains **why A*** is smarter than BFS, DFS, or even UCS.

Let's focus on the logic, not the code.

What is a Heuristic?

Imagine you're in Accra and want to drive to Kumasi.

Before starting the trip, you don't know:



```
class BestFirstSearch(SearchAlgorithm):
    """Generic best-first search. Subclasses define the evaluation
    function f(n)."""

    algorithm_name = "BestFirst"

    def __init__(
        self,
        heuristic: Callable[[Tuple[int, int], Tuple[int, int]], float] =
        zero_heuristic,
    ):
        Afficher plus ▾
```

Excellent. This is the **heart of the entire lab**.

Everything before this section was preparation.

This method is where:

- UCS works
- Greedy works
- A* works
- Weighted A* works

The beautiful thing is that **all four algorithms use the same search code**.

Best-First Search Pseudocode Guide

Study this pseudocode before completing the code cell below.

Compare it line by line with the BFS pseudocode from Part A and find the three differences (priority queue, reached as a dictionary with cost comparison, late goal test).

```
BEST-FIRST-SEARCH(problem, f)
  node ← NODE(problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a dictionary {problem.INITIAL : node}
```

Afficher plus ▾

Based on the pseudocode you pasted, the implementation should look like this:

Python

```
def search(self, problem: Problem) -> SearchResult:
```

```
    # 1. Create initial node
    node = Node(problem.initial_state())

    # 2. Create priority queue frontier
    frontier = PriorityQueue()
    frontier.push(
        self.evaluation(node, problem),
        node
```



▶ Exécuter



This one is much simpler than TODO 4.

Remember the formula from the lecture:

Greedy Best-First Search

$f(n) = h(n)$



The parent class already gives you:

Python

```
self.h(node, problem)
```



▶ Exécuter

and `h()` already computes:

Python

```
self.heuristic(node.state, problem.goal)
```



▶ Exécuter

So Greedy does **not** use:

$g(n)$ = path cost



It only uses:

$h(n)$ = estimated distance to goal



Therefore the evaluation function is simply

Why multiply the heuristic?

Suppose:

Node	$g(n)$	$h(n)$
A	8	2
B	4	5

Regular A*:

$$\begin{aligned} A &= 8 + 2 = 10 \\ B &= 4 + 5 = 9 \end{aligned}$$



Chooses:

B



Weighted A* with:

$$W = 2$$



gives:

$$\begin{aligned} A &= 8 + 2 \times 2 = 12 \\ B &= 4 + 2 \times 5 = 14 \end{aligned}$$



```

def __init__(self, problem):
    self.problem = problem
    self.algorithms = [
        GreedyBestFirstSearch(manhattan_distance),
        AStarSearch(manhattan_distance),
        WeightedAStarSearch(manhattan_distance, weight=2),
        UniformCostSearch(),
    ]

    res = algo.search(check_problem)
    assert res.status == "success", f"(res.algorithm) failed to find a path"
    assert res.solution_cost == 4, (
        f"(res.algorithm) returned cost {res.solution_cost}, expected 4"
    )

Cell In[14], line 72
    68
    69
    70     nodes_expanded += 1
    71
    72     for child in self.expand(problem, node):
    73         s = child.state
    74         if s not in reached or child.path_cost < reached[s].path_cost:
    75             reached[s] = child

Cell In[7], line 147
    143     # yield NODE(state=s_prime, parent=node, action=action, path_cost=cost)
    144     s = node.state
    145
    146     # In this loop, each valid move has a cost of 1.
    147     # Return 1.
    148     raise NotImplementedError("Complete GridProblem.action_cost")

NotImplementedError: Complete GridProblem.action_cost

```

```

check_grid = [
    [0, 0, 0],
    [1, 1, 0],
    [0, 0, 0],
]
check_problem = GridProblem(check_grid, start=(0, 0), goal=(2, 2))

```

```

for algo in [
    GreedyBestFirstSearch(manhattan_distance),
    AStarSearch(manhattan_distance),
    WeightedAStarSearch(manhattan_distance, weight=2),
]

```

Afficher plus ▾



The error is not coming from Greedy, A*, or Weighted A*.

Your `WeightedGridProblem.action_cost()` is actually fine.

The problem is that the error message says:

```
NotImplementedError: Complete GridProblem.action_cost
```



not

```
NotImplementedError: Complete WeightedGridProblem.action_cost
```



So the failing method is a different one.

Look for something like:

Python



Exécuter

```
class GridProblem(Problem):

    def action_cost(
        self,
        state,
        action,
        next_state,
    ):
        # TODO ...
        raise NotImplementedError("Complete GridProblem.action_cost")
```

The self-check is creating:



```
class GridProblem(Problem):
    def __init__(
        self,
        grid: List[List[int]],
        start: Tuple[int, int],
        goal: Tuple[int, int],
    ):
        """
        grid:
            2D list where 0 = free cell and 1 = obstacle.
```

Afficher plus ▾

I found the issue.

Your traceback is caused by:

Python



Exécuter

```
def action_cost(
    self,
    state: Tuple[int, int],
    action: str,
    next_state: Tuple[int, int],
) -> float:
    # TODO 4:
    # In this Lab, each valid move has a cost of 1.
    # Return 1.
    raise NotImplementedError("Complete GridProblem.action_cost")
```



```
class BestFirstSearch(SearchAlgorithm):
    """Generic best-first search. Subclasses define the evaluation
    function f(n)."""

    algorithm_name = "BestFirst"

    def __init__(
        self,
        heuristic: Callable[[Tuple[int, int], Tuple[int, int]], float] =
        zero_heuristic,
    ):
        Afficher plus ▾
```



The problem is right here:

```
Python
return SearchResult(
    status="success",
    solution=node.solution(), # <-- ERROR
    path_cost=node.path_cost,
    ...
)
```



▶ Exécuter

Your traceback already told us:



where to make the change

In the `BestFirstSearch.search()` method that you pasted earlier.

Find this block:

```
Python
if problem.is_goal(node.state):
    return SearchResult(
        status="success",
        solution=node.solution(),
        path_cost=node.path_cost,
        nodes_expanded=nodes_expanded,
        max_frontier_size=max_frontier_size,
        reached_states=len(reached),
        algorithm=self.algorithm_name,
    )
```



▶ Exécuter

Replace the entire block with:

```
Python
if problem.is_goal(node.state):
    return SearchResult(
        algorithm=self.algorithm_name,
        status="success",
        solution=node,
        nodes_expanded=nodes_expanded,
        max_frontier_size=max_frontier_size,
```



▶ Exécuter



TODO 8 should be:

Python



▶ Exécuter

```
def inflated_heuristic(factor: float):  
  
    def h(state, goal):  
        return factor * manhattan_distance(state, goal)  
  
    return h
```

Why this works

If you call:

Python



▶ Exécuter

```
inflated = inflated_heuristic(3.0)
```

then Python creates a new function:

Python



▶ Exécuter

```
h(state, goal)
```

that remembers:



	Algorithm	Status	Solution depth	Solution cost	Nodes expanded	Max frontier	Reached states
0	A* admissible (uniform map)	success	18	18	18	5	23
1	A* inflated x3 (uniform map)	success	18	18	18	5	23
2	A* admissible (mild turbulence)	success	10	10	29	10	85
3	A* inflated x3 (mild turbulence)	success	6	12	6	13	19

Record your observations in a markdown cell here:

On which map(s) did the inflated heuristic return a suboptimal path? By how much?

How many fewer nodes did it expand?

In one or two sentences: when would you, as the drone's engineer, deliberately accept an inadmissible heuristic?

Based on your results table, you can write:

 Modifier



Observations

1. On which map(s) did the inflated heuristic return a suboptimal path? By how much?

The inflated heuristic returned a suboptimal path on the **mild turbulence map**. The admissible A* heuristic found a path with a solution cost of **10**, while the inflated heuristic ($3 \times$ Manhattan distance) found a path with a solution cost of **12**. Therefore, the inflated heuristic produced a path that was **2 cost units more expensive** than the optimal path.

On the **uniform map**, both the admissible and inflated heuristics found the same optimal solution cost of **18**, so no suboptimality occurred there.



Dictate Ctrl + Shift + D